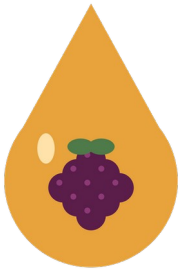
[NOMBRE DE LA INSTITUCIÓN / UNIVERSIDAD]

[Facultad / Carrera]



MORA & MIEL

Pastelería artesanal

Integración de una API REST pública: Recetario dulce

Parte 3 · Consumo de TheMealDB con http, Provider y manejo de estados

Estudiante	Cinthia Marisol Guamán Montaño
Asignatura	Desarrollo de Aplicaciones Móviles
Docente	[Nombre del docente]
Repositorio	[https://github.com/Cintygm/<repositorio>]
Fecha de entrega	[dd/mm/2026]

Sección I: Selección y descripción de la API pública

Dato	Detalle
Nombre	TheMealDB: base de datos abierta de recetas de cocina
Documentación	https://www.themealdb.com/api.php
Endpoint 1 (listado)	GET https://www.themealdb.com/api/json/v1/1/filter.php?c=Dessert
Endpoint 2 (detalle)	GET https://www.themealdb.com/api/json/v1/1/lookup.php?i={idMeal}
Autenticación	Ninguna. El "1" de la ruta es la clave pública de prueba que la API publica para todos; no hay registro, sesión ni token
Formato	JSON, codificado en UTF-8

Justificación de la elección

Mora & Miel es una pastelería que acepta **reservas y pedidos personalizados**. Muchas veces el cliente no sabe qué pedir. El nuevo **Recetario dulce** muestra postres de todo el mundo, con foto, ingredientes y preparación, para que el cliente se inspire y la pastelera tenga una referencia clara.

- **Relación directa con el negocio:** la API filtra por la categoría `Dessert`, así que solo se muestran postres.
- **Datos visuales:** cada receta trae una fotografía, ideal para una cuadrícula de tarjetas atractiva.
- **Dos endpoints encadenados:** el listado entrega lo mínimo (id, nombre, foto) y el detalle se pide solo cuando el usuario toca una receta, lo que ahorra datos móviles.
- **Cumple las restricciones:** es REST, gratuita y no requiere inicio de sesión.

Estructura del JSON consumido

Listado: GET filter.php?c=Dessert

```
{
  "meals": [
    {
      "strMeal": "Apple Frangipan Tart",
      "strMealThumb": "https://www.themealdb.com/images/media/meals/wxywrq1468235067.jpg",
      "idMeal": "52768"                // el id llega como texto
    }
    // ... un objeto por postre
  ]
}
```

Detalle: GET lookup.php?i=52768 (extracto)

```
{
  "meals": [
    {
      "idMeal": "52768",
      "strMeal": "Apple Frangipan Tart",
      "strCategory": "Dessert",
      "strArea": "British",
      "strInstructions": "Preheat the oven to 200C...\r\nPut the biscuits in...",
      "strMealThumb": "https://www.themealdb.com/images/media/meals/wxywrq1468235067.jpg",
      "strYoutube": "https://www.youtube.com/watch?v=rp8Slv4INLk",
      "strIngredient1": "digestive biscuits",  "strMeasure1": "175g/6oz",
      "strIngredient2": "butter",              "strMeasure2": "75g/3oz",
      // ... hasta strIngredient20 / strMeasure20 (vacíos o null si no se usan)
```

```
    }
  ]
}
```

Dos detalles del JSON obligan a un mapeo cuidadoso. Primero, `idMeal` llega como texto y se convierte a `int`. Segundo, los ingredientes no vienen en una lista, sino en 20 pares de campos numerados que hay que recorrer, limpiar y descartar cuando están vacíos. Cuando no encuentra una receta, la API responde `{"meals": null}`, y ese caso también se maneja.

Sección II: Arquitectura y manejo de estados

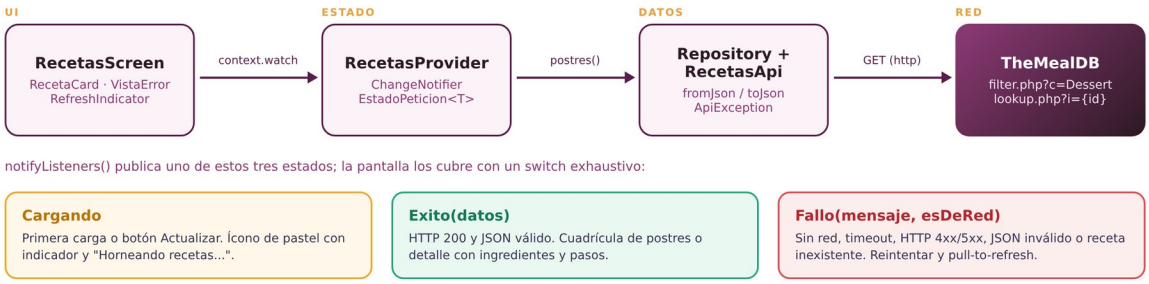


Figura 1. Capas del recetario y estados de la petición.

El recetario sigue la arquitectura por capas de la Actividad 2 con el patrón **Provider + ChangeNotifier**. La regla es que la interfaz nunca importa `http` ni lee JSON:

Capa	Clases	Responsabilidad
Presentación	RecetasScreen, RecetaDetalleScreen, RecetaCard, VistaCargando, VistaError	Dibujar según el estado y enviar acciones (cargar, buscar, reintentar).
Estado	RecetasProvider, RecetaDetalleProvider, EstadoPeticion<T>	Ejecutar la petición, convertir el resultado en un estado y notificar a la UI.
Repositorio	RecetasRepository	Única puerta de entrada a los datos. Permite cambiar de API sin tocar la UI.
Fuente remota	RecetasApi, ApiException	GET con http, timeout de 10 s, traducción de errores y mapeo JSON → modelos.

Estados de la petición

Se creó una sola clase `sealed` genérica, `EstadoPeticion<T>`, que reutilizan el listado (`T = List<RecetaResumen>`) y el detalle (`T = RecetaDetalle`). Como es `sealed`, el compilador exige que cada `switch` de la interfaz cubra los tres casos.

Estado	Cuándo	Qué ve el usuario
Cargando	Primera carga o botón Actualizar	Ícono de pastel con indicador circular y "Horneando recetas..."
Exito(datos)	HTTP 200 y JSON válido	Cuadrícula de recetas con foto o detalle con ingredientes y pasos

Fallo(mensaje, esDeRed)	Sin red, timeout, HTTP 4xx/5xx, JSON inválido o receta inexistente	"Estás sin conexión" (Wi-Fi tachado) o "Algo salió mal", con botón Reintentar

Decisiones de diseño

- **Provider por pantalla de detalle.** RecetaDetalleProvider se crea al abrir una receta y se destruye al salir. Si el usuario vuelve atrás antes de que llegue la respuesta, el provider no notifica a una pantalla que ya no existe.
- **Sin parpadeo al refrescar.** Con pull-to-refresh y datos ya visibles, el estado no vuelve a Cargando; el RefreshIndicator muestra el progreso.
- **Sin peticiones duplicadas.** Si ya hay una carga en curso, una segunda llamada se ignora.
- **Búsqueda local.** El buscador filtra la lista ya descargada, sin consumir datos.
- **Imágenes resilientes.** Las fotos usan la miniatura /preview de la API, con indicador propio de carga y un ícono de respaldo si fallan.

Sección III: Extractos de código clave

3.1 Consumo HTTP y mapeo del modelo JSON

Petición GET asíncrona y traducción de errores (extracto)

```
// lib/data/sources/recetas_api.dart
static const String baseUrl = 'https://www.themealdb.com/api/json/v1/1';

Future<List<RecetaResumen>> obtenerPostres() async {
  final json = await _get('/filter.php?c=Dessert');
  final meals = json['meals'] as List<dynamic>?;
  if (meals == null) return const [];
  return meals
      .map((e) => RecetaResumen.fromJson(e as Map<String, dynamic>))
      .toList(growable: false);
}

Future<Map<String, dynamic>> _get(String ruta) async {
  final http.Response respuesta;
  try {
    respuesta = await _cliente.get(Uri.parse('$baseUrl$ruta')).timeout(tiempoMaximo);
  } on SocketException {
    throw const ApiException(TipoErrorApi.sinConexion,
        'Sin conexión a internet. Revisa tu Wi-Fi o datos móviles.');
  } on TimeoutException {
    throw const ApiException(TipoErrorApi.sinConexion,
        'El servidor tardó demasiado en responder.');
  }
  if (respuesta.statusCode != 200) {
    throw ApiException(TipoErrorApi.servidor,
        'El servidor respondió con el código ${respuesta.statusCode}.');
  }
  return jsonDecode(utf8.decode(respuesta.bodyBytes)) as Map<String, dynamic>;
}
```

Mapeo de los 20 campos de ingredientes a una lista tipada

```
// lib/data/models/receta.dart
factory RecetaDetalle.fromJson(Map<String, dynamic> json) {
  final ingredientes = <Ingrediente>[];
  for (var i = 1; i <= maxIngredientes; i++) {          // strIngredient1..20
    final nombre = (json['strIngredient$i'] as String?)?.trim() ?? '';
    if (nombre.isEmpty) continue;                       // descarta vacíos y null
    final medida = (json['strMeasure$i'] as String?)?.trim() ?? '';
    ingredientes.add(Ingrediente(nombre, medida));
  }
```

```
  return RecetaDetalle(
    id: int.parse(json['idMeal'] as String),            // "52768" -> 52768
    nombre: json['strMeal'] as String,
    categoria: (json['strCategory'] as String?) ?? 'Dessert',
    origen: (json['strArea'] as String?) ?? '',
    instrucciones: (json['strInstructions'] as String?)?.trim() ?? '',
    foto: (json['strMealThumb'] as String?) ?? '',
    ingredientes: List.unmodifiable(ingredientes),
  );
}

Map<String, dynamic> toJson() => {
  'idMeal': '$id', 'strMeal': nombre, 'strCategory': categoria,
  'strArea': origen, 'strInstructions': instrucciones, 'strMealThumb': foto,
  for (var i = 0; i < ingredientes.length; i++) ...{
    'strIngredient${i + 1}': ingredientes[i].nombre,
    'strMeasure${i + 1}': ingredientes[i].medida,
  },
};
```

3.2 Manejo de estados en la interfaz

Estados sealed y provider del listado

```
// lib/providers/estado_peticion.dart
sealed class EstadoPeticion<T> { const EstadoPeticion(); }
final class Cargando<T> extends EstadoPeticion<T> { const Cargando(); }
final class Exito<T> extends EstadoPeticion<T> { const Exito(this.datos); final T datos; }
final class Fallo<T> extends EstadoPeticion<T> {
  const Fallo(this.mensaje, {this.esDeRed = false});
  final String mensaje;
  final bool esDeRed;
}

// lib/providers/recetas_provider.dart
Future<void> cargar({bool refrescar = false}) async {
  if (_actualizando) return;                    // evita peticiones duplicadas
  _actualizando = true;
  if (!refrescar || _estado is! Exito) _estado = const Cargando();
  notifyListeners();
  try {
    _estado = Exito(await _repo.postres());
  } on ApiException catch (e) {
    _estado = Fallo(e.mensaje, esDeRed: e.esDeRed);
  } catch (_) {
    _estado = const Fallo('Ocurrió un error inesperado. Intenta de nuevo.');
  } finally {
    _actualizando = false;
    notifyListeners();
  }
}
```

Interfaz: switch exhaustivo sobre el estado

```
// lib/screens/recetas_screen.dart
final recetas = context.watch<RecetasProvider>();

RefreshIndicator(
  onRefresh: () => recetas.cargar(refrescar: true),          // pull-to-refresh
  child: CustomScrollView(
    physics: const AlwaysScrollableScrollPhysics(),
    slivers: [
      SliverToBoxAdapter(child: _Encabezado(recetas: recetas)),
      ...switch (recetas.estado) {                            // exhaustivo
        Cargando() => [const SliverFillRemaining(hasScrollBody: false,
            child: VistaCargando(mensaje: 'Horneando recetas...'))],
        Exito() => _contenido(context, recetas.visibles),     // cuadrícula
        Fallo(:final mensaje, :final esDeRed) => [SliverFillRemaining(
            hasScrollBody: false,
            child: VistaError(mensaje: mensaje, esDeRed: esDeRed,
                onReintentar: () => recetas.cargar()))],
      },
    ],
```

```
  ),
)
```

Sección IV: Evidencias de pruebas (QA) y enlace al repositorio

4.1 Repositorio público

URL: `[https://github.com/Cintygm/<repositorio>]`

La funcionalidad se desarrolló en la rama `feature/api-integration`, se integró en `develop` y luego en `main` con `--no-ff`, y se etiquetó como `v3.0.0`. El README tiene la sección "Parte 3: Integración de API REST pública" con los endpoints, el JSON y las instrucciones de ejecución.

Commit	Contenido
feat(api)	Consumo de TheMealDB con http, ApiException y modelos con fromJson/toJson
feat(estado)	EstadoPeticion<T> sealed y providers del listado y del detalle
feat(ui)	Recetario con cuadrícula, búsqueda, detalle y pull-to-refresh
test(api)	Mapeo JSON, errores HTTP y transiciones de estado con MockClient
docs	README e informe de la Parte 3
chore(release)	Mora & Miel 3.0.0

4.2 Casos de prueba

Caso	Pasos	Resultado esperado
Cargando	Abrir el Recetario (emulador con red lenta: Cellular → GSM)	"Horneando recetas..." con indicador
Éxito	Abrir el Recetario con internet	Cuadrícula de postres con foto y total en el encabezado
Búsqueda	Escribir "cake"	Se filtra sin nuevas peticiones
Pull-to-refresh	Deslizar hacia abajo	Indicador circular; la cuadrícula se mantiene
Detalle	Tocar una receta	Foto, categoría, origen, ingredientes y pasos
Error de red	Modo avión → Actualizar	"Estás sin conexión" y botón Reintentar
Recuperación	Quitar modo avión → Reintentar	Vuelve la cuadrícula
Pruebas unitarias	flutter test test/recetas_test.dart	11 pruebas: mapeo, sin red, HTTP 500, meals null, estados y búsqueda

4.3 Capturas de funcionamiento

Reemplazar cada recuadro por la captura real del emulador o dispositivo.

Captura 1: Cargando
Pegar aquí: "Horneando recetas..."

Figura 2. Estado Cargando.

Captura 2: Éxito
Pegar aquí: Cuadrícula de postres

Figura 3. Datos de la API cargados.

Captura 3: Error de red
Pegar aquí: Modo avión activado

Figura 4. Manejo del error de red.

Captura 4: Detalle de receta
Pegar aquí: Ingredientes y preparación

Figura 5. Segunda petición: detalle.

Captura 5: Pull-to-refresh
Pegar aquí: Indicador sobre la cuadrícula

Figura 6. Actualización manual.

Sección V: Conclusiones

5.1 La red es lenta e impredecible: por eso todo es asíncrono

Descargar el recetario puede tardar desde unos milisegundos hasta varios segundos. Gracias a Future y async/await, la petición corre sin bloquear el hilo de la interfaz: mientras se espera, la app sigue animando el indicador, respondiendo al scroll y permitiendo volver atrás. Además, cada imagen se descarga de forma independiente, con su propio indicador, así que una foto lenta no detiene el resto de la cuadrícula. El timeout de 10 segundos asegura que ninguna espera sea infinita.

5.2 El error es un estado más, no un caso raro

En un celular la conexión se pierde todo el tiempo: el ascensor, el bus, un Wi-Fi sin internet. Por eso el error se modeló con la misma importancia que la carga y el éxito, dentro de una clase sealed que el compilador obliga a manejar. Traducir excepciones técnicas (SocketException, código 500, JSON inválido, receta inexistente) a mensajes en español, y distinguir "no tienes conexión" de "falló el servidor", le dice al usuario qué pasó y qué puede hacer: reintentar o deslizar para actualizar.

5.3 Desacoplar permite probar sin internet

Como la interfaz solo depende de los providers y la fuente de datos recibe el cliente HTTP por constructor, se pudo probar cada escenario con un MockClient que simula respuestas correctas, errores 500, falta de red o JSON incompletos. El mismo diseño permitiría cambiar TheMealDB por otra API, o el paquete http por dio, sin modificar ninguna pantalla.

5.4 Conclusión general

Pedir datos a una API ocupa una sola línea de código. Lo que hace confiable a una aplicación es todo lo que rodea esa línea: modelos que validan y limpian el JSON, estados explícitos para cada resultado posible, una interfaz que siempre informa qué está pasando y una ruta clara para recuperarse de los errores. Con esta parte, Mora & Miel pasa de mostrar solo su catálogo propio a ofrecer contenido vivo, que ayuda a sus clientes a imaginar su próximo pedido.